COMPUTING LAB - II
ADVANCED SYSTEM SOFTWARE LAB - CSE 5261
MTech - CSE
Week - 6

Name : Rajat Jain
Reg No : 200913010

Lab Exercises:

1) Write a Recursive Descent Parsing For C grammar.

Code

```
print("Recursive Descent Parsing For C grammar\n")
global s
program = input("Enter the program to be parsed\n")
s=list(program)
global i
i=0
def match(a):
    global s
    global i
    if(i>=len(s)):
        return False
    j = "".join(s[i:i+len(a)])
    if(j == a):
        print(f"{j} == {a} , Matched")
        i = i + len(a)
        return True
    else:
        return False


def Program():
    if(match("main(){")):
        if(declarations()):
            if(statement_list()):
                if(declarations()):
                    if(match("}")):
```

```
                return True
            else:
                return False
        else:
            return False
    else:
        return False
else:
    return False


def declarations():
    if(data_type()):
        if(identifier_list()):
            if(match(";")):
                if(declarations()):
                    return True
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return True


def data_type():
    if(match("int")):
        return True
    elif(match("char")):
        return True
    elif(match("float")):
        return True
    else:
        return False

def identifier_list():
    if(id()):
        if(X()):
            return True
```

```
        else:
            return False
    else:
        return False
def X():
    if(match(",")):
        if(identifier_list()):
            return True
        else:
            return False
    elif(match("[")):
        if(number()):
            if(match("]")):
                if(Y()):
                    return True
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return True

def Y():
    if(match(",")):
        if(identifier_list()):
            return True
        else:
            return False
    else:
        return True


def statement_list():
    if(statement()):
        if(statement_list()):
            return True
        else:
            return False
    else:
        return True
```

```
def statement():
    if(assign_stat()):
        if(match(";")):
            return True
        else:
            return False
    elif(decision_stat()):
        return True
    elif(looping_stat()):
        return True
    else:
        return False

def assign_stat():
    if(id()):
        if(match("=")):
            if(expn()):
                return True
            else:
                return False
        else:
            return False
    else:
        return False

def expn():
    if(simple_expn()):
        if(eprime()):
            return True
        else:
            return False
    else:
        return False

def eprime():
    if(relop()):
        if(simple_expn()):
            return True
        else:
            return False
    else:
        return True
```

```
def simple_expn():
    if(term()):
        if(seprime()):
            return True
        else:
            return False
    else:
        return False

def seprime():
    if(addop()):
        if(term()):
            if(seprime()):
                return True
            else:
                return False
        else:
            return False
    else:
        return True

def term():
    if(factor()):
        if(tprime()):
            return True
        else:
            return False
    else:
        return False

def tprime():
    if(mulop()):
        if(factor()):
            if(tprime()):
                return True
            else:
                return False
        else:
            return False
    else:
        return True

def factor():
```

```
    if(id()):
        return True
    elif(number()):
        return True
    else:
        return False
def decision_stat():
    if(match("if(")):
        if(expn()):
            if(match(")\{")):
                if(statement_list()):
                    if(match("\}")):
                        if(dprime()):
                            return True
                        else:
                            return False
                    else:
                        return False
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False

def dprime():
    if(match("else\{")):
        if(statement_list()):
            if(match("\}")):
                return True
            else:
                return False
        else:
            return False
    else:
        return True

def looping_stat():
    if(match("while(")):
        if(expn()):
            if(match(")\{")):
```

```
            if(statement_list()):
                if(match("}")):
                    return True
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False
elif(match("for(")):
    if(assign_stat()):
        if(match(";")):
            if(expn()):
                if(match(";")):
                    if(assign_stat()):
                        if(match(")\{")):
                            if(statement_list()):
                                if(match("}")):
                                    return True
                                else:
                                    return False
                            else:
                                return False
                        else:
                            return False
                    else:
                        return False
                else:
                    return False
            else:
                return False
        else:
            return False
    else:
        return False

def relop():
    if(match("==")):
        return True
```

```
    elif(match("!=")):
        return True
    elif(match(">")):
        return True
    elif(match("<")):
        return True
    else:
        return False
def addop():
    if(match("+")):
        return True
    elif(match("-")):
        return True
    else:
        return False

def mulop():
    if(match("*")):
        return True
    elif(match("/")):
        return True
    elif(match("%")):
        return True
    else:
        False

def id():
    if(match('a')):
        return True
    elif(match('b')):
        return True
    elif(match('c')):
        return True
    elif(match('d')):
        return True
    elif(match('e')):
        return True
    elif(match('j')):
        return True
    else:
        return False
```

```
def number():
    if(match('0')):
        return True
    elif(match('1')):
        return True
    elif(match('2')):
        return True
    elif(match('3')):
        return True
    elif(match('4')):
        return True
    elif(match('5')):
        return True
    elif(match('6')):
        return True
    elif(match('7')):
        return True
    elif(match('8')):
        return True
    elif(match('9')):
        return True
    else:
        return False

if(Program()):
    if(i == len(s)):
        print("Accepted")
    else:
        print("Not Accepted")
else:
    print("Not Accepted")
```

Output

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){}
main(){ == main(){ , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta;}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
; == ; , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta,b;b=a;}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
, == , , Matched
b == b , Matched
; == ; , Matched
b == b , Matched
= == = , Matched
a == a , Matched
; == ; , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta,b,c[5];floatd;c=b+a;d=c;}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
, == , , Matched
b == b , Matched
, == , , Matched
c == c , Matched
[ == [ , Matched
5 == 5 , Matched
] == ] , Matched
; == ; , Matched
float == float , Matched
d == d , Matched
; == ; , Matched
c == c , Matched
= == = , Matched
b == b , Matched
+ == + , Matched
a == a , Matched
; == ; , Matched
d == d , Matched
= == = , Matched
c == c , Matched
; == ; , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta;a=2;for(j=0;j<5;j=j+1){a=a+2;}}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
; == ; , Matched
a == a , Matched
= == = , Matched
2 == 2 , Matched
; == ; , Matched
for( == for( , Matched
j == j , Matched
= == = , Matched
0 == 0 , Matched
; == ; , Matched
j == j , Matched
< == < , Matched
5 == 5 , Matched
; == ; , Matched
j == j , Matched
= == = , Matched
j == j , Matched
+ == + , Matched
1 == 1 , Matched
){ == ){ , Matched
a == a , Matched
= == = , Matched
a == a , Matched
+ == + , Matched
2 == 2 , Matched
; == ; , Matched
} == } , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta;a=6;if(a>4){a=a-1;}else{a=a+1;}}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
; == ; , Matched
a == a , Matched
= == = , Matched
6 == 6 , Matched
; == ; , Matched
if( == if( , Matched
a == a , Matched
> == > , Matched
4 == 4 , Matched
){ == ){ , Matched
a == a , Matched
= == = , Matched
a == a , Matched
- == - , Matched
1 == 1 , Matched
; == ; , Matched
} == } , Matched
else{ == else{ , Matched
a == a , Matched
= == = , Matched
a == a , Matched
+ == + , Matched
1 == 1 , Matched
; == ; , Matched
} == } , Matched
} == } , Matched
Accepted
```

```
Recursive Desent Parsing For C grammar

Enter the program to be parsed
main(){inta;a=5;while(a>2){a=a-1;}}
main(){ == main(){ , Matched
int == int , Matched
a == a , Matched
; == ; , Matched
a == a , Matched
= == = , Matched
5 == 5 , Matched
; == ; , Matched
while( == while( , Matched
a == a , Matched
> == > , Matched
2 == 2 , Matched
){ == ){ , Matched
a == a , Matched
= == = , Matched
a == a , Matched
- == - , Matched
1 == 1 , Matched
; == ; , Matched
} == } , Matched
} == } , Matched
Accepted
```

2. Write a python program to generate tokens for the perl script given below.

```
#! /usr/bin/perl
#get total number of arguments passed.
$n = scalar (@_);
$sum = 0;
foreach $item(@_) {
$sum += $item;
} $
average =
$
sum +
$
n;
```

#! Represents path which has to be ignored.
followed by any character other than ! represents comments.
$n followed by any identifier should be treated as a single token.
Scalar, foeach are considered as keywords.
@_, += are treated as single tokens.
Remaining symbols are tokenized accordingly.

<u>Code</u>

```
from ply import lex
import re


tokens = [
    # Literals (identifier, integer constant, float constant, string constant, char
const)
    'ID', 'TYPEID', 'INTEGER', 'FLOAT', 'STRING',
'CHARACTER','COMMENT',

    # Operators (+,-,*,/,%,|,&,~,^,<<,>>, ||, &&, !, <, <=, >, >=, ==, !=)
    'PLUS', 'MINUS', 'TIMES', 'DIVIDE', 'MODULO',
    'OR', 'AND', 'NOT', 'XOR', 'LSHIFT', 'RSHIFT',
    'LOR', 'LAND', 'LNOT',
    'LT', 'LE', 'GT', 'GE', 'EQ', 'NE',

    # Assignment (=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, |=)
    'EQUALS', 'TIMESEQUAL', 'DIVEQUAL', 'MODEQUAL', 'PLUSEQUAL',
'MINUSEQUAL',
```

```
    'LSHIFTEQUAL','RSHIFTEQUAL', 'ANDEQUAL', 'XOREQUAL',
'OREQUAL',

    # Increment/decrement (++,--)
    'INCREMENT', 'DECREMENT',

    # Structure dereference (->)
    'ARROW',

    # Ternary operator (?)
    'TERNARY',

    # Delimeters ( ) [ ] { } , . ; :
    'LPAREN', 'RPAREN',
    'LBRACKET', 'RBRACKET',
    'LBRACE', 'RBRACE',
    'COMMA', 'PERIOD', 'SEMI', 'COLON',

    # Ellipsis (...)
    'ELLIPSIS',

    # Keywords
    'KEYWORD',

    # Parameter Array
    'PARAMETERARRAY',

    # Addition Assignment Operator
    'ADDITIONASSIGNMENT'
]

# Operators
t_PLUS            = r'\+'
t_MINUS           = r'-'
t_TIMES           = r'\*'
t_DIVIDE          = r'/'
t_MODULO          = r'%'
t_OR              = r'\|'
t_AND             = r'&'
t_NOT             = r'~'
t_XOR             = r'\^'
t_LSHIFT          = r'<<'
t_RSHIFT          = r'>>'
```

```
t_LOR           = r'\|\|'
t_LAND          = r'&&'
t_LNOT          = r'!'
t_LT            = r'<'
t_GT            = r'>'
t_LE            = r'<='
t_GE            = r'>='
t_EQ            = r'=='
t_NE            = r'!='

# Assignment operators

t_EQUALS        = r'='
t_TIMESEQUAL    = r'\*='
t_DIVEQUAL      = r'/='
t_MODEQUAL      = r'%='
t_PLUSEQUAL     = r'\+='
t_MINUSEQUAL    = r'-='
t_LSHIFTEQUAL   = r'<<='
t_RSHIFTEQUAL   = r'>>='
t_ANDEQUAL      = r'&='
t_OREQUAL       = r'\|='
t_XOREQUAL      = r'\^='

# Increment/decrement
t_INCREMENT     = r'\+\+'
t_DECREMENT     = r'--'

# ->
t_ARROW         = r'->'

# ?
t_TERNARY       = r'\?'

# Delimeters
t_LPAREN        = r'\('
t_RPAREN        = r'\)'
t_LBRACKET      = r'\['
t_RBRACKET      = r'\]'
t_LBRACE        = r'\{'
t_RBRACE        = r'\}'
t_COMMA         = r','
#t_PERIOD       = r'\.'
```

```
t_SEMI            = r';'
t_COLON           = r':'
t_ELLIPSIS        = r'\.\.\.'

# Identifiers
t_ID = r'\$[A-Za-z_][A-Za-z0-9_]*'


# Integer literal
t_INTEGER = r'\d+([uU]|[lL]|[uU][lL]|[lL][uU])?'

# Floating literal
t_FLOAT = r'((\d+)(\.\d+)(e(\+|-)?(\d+))? | (\d+)e(\+|-)?(\d+))([lL]|[fF])?'

# String literal
t_STRING = r'\"([^\\\n]|(\\.))*?\"'

# Character constant 'c' or L'c'
t_CHARACTER = r'(L)?\'([^\\\n]|(\\.))*?\'

key_words = ['scalar','foreach']

t_KEYWORD = r'|'.join(key_words)

t_COMMENT = r'\#.*'

#Parameter Array and Addition Assignment

t_PARAMETERARRAY = r'@_'
t_ADDITIONASSIGNMENT = r'\+='

# Compute column.
 #     input is the input text string
 #     token is a token instance
def find_column(input, token):
    line_start = input.rfind('\n', 0, token.lexpos) + 1
    return (token.lexpos - line_start) + 1


def t_newline(t):
  r'\n+'
  t.lexer.lineno += len(t.value)
```

```
t_ignore = ' \t'


def t_error(t):
  print(f"Illegal character {t.value[0]}")
  t.lexer.skip(1)

lex.lex()

lines = []
with open('perlprog.pl','r') as file:
	for line in file:
		if not line.startswith('#!'):
			lines.append(line)

data = ''.join(lines)

print(data)

generated_tokens = []

lex.input(data)
#Tokenize
while 1:
	tok = lex.token()
	if not tok: break
	print(tok)
	print(f"Column : {find_column(data,tok)} Row : {tok.lineno} Type :
{tok.type} Value : {tok.value}")
	token = f'<{tok.lineno},{find_column(data,tok)},{tok.value},{tok.type}>'
	generated_tokens.append(token)
output = '\n'.join(generated_tokens)
print(output)
with open('output.txt','w') as file:
	file.write(output)
	file.write(f'\nTotal number of tokens generated =
{len(generated_tokens)}')
```

Input

```
#! /usr/bin/perl
#get total number of arguments passed.
$n = scalar(@_);
$sum = 0;
foreach $item(@_) {
$sum += $item;
}
$average = $sum + $n;
```

Output

```
<1,1,#get total number of arguments passed.,COMMENT>
<2,1,$n,ID>
<2,4,=,EQUALS>
<2,6,scalar,KEYWORD>
<2,12,(,LPAREN>
<2,13,@_,PARAMETERARRAY>
<2,15,),RPAREN>
<2,16,;,SEMI>
<3,1,$sum,ID>
<3,6,=,EQUALS>
<3,8,0,INTEGER>
<3,9,;,SEMI>
<4,1,foreach,KEYWORD>
<4,9,$item,ID>
<4,14,(,LPAREN>
<4,15,@_,PARAMETERARRAY>
<4,17,),RPAREN>
<4,19,{,LBRACE>
<5,1,$sum,ID>
<5,6,+=,PLUSEQUAL>
<5,9,$item,ID>
<5,14,;,SEMI>
<6,1,},RBRACE>
<7,1,$average,ID>
<7,10,=,EQUALS>
<7,12,$sum,ID>
<7,17,+,PLUS>
<7,19,$n,ID>
<7,21,;,SEMI>
Total number of tokens generated = 29
```